FedRAS: Network-Conditioned Rank Allocation for Federated LoRA

Wenyu Ma[a,b,c], Tao Hu[a,b,c,*], Xinlei Liu[a,b,c]

[a]*Information Engineering University, Zhengzhou 450001, China*
[b]*Key Laboratory of Cyberspace Endogenous Safety and Security of Henan Province, Zhengzhou 450001, China*
[c]*Key Laboratory of Cyberspace Security, Ministry of Education of China, Zhengzhou 450001, China*

Abstract

Federated LoRA reduces update size, but a fixed rank cannot adapt to time-varying links. We formulate rank selection and client admission as a joint network scheduling problem and call the resulting method FedRAS. It estimates a delivery probability and an adaptation capacity for each client–rank pair, maximizes the expected useful updates under byte-budget and participation constraints, solves the assignment with a dynamic program, and aggregates heterogeneous ranks with frozen-factor rank masking. All payloads are accounted at their real serialized sizes: a frozen-A client uploads only the trainable B factor, 0.090 MB per round at rank 16, against 5.29 MB for a full-model update.

Experiments on non-IID CIFAR-10, CIFAR-100, and MNIST with a 500-client scheduler simulation use deadlines recalibrated to these real payloads. At the nominal deadline, rank selection reduces to fixed rank-16 behavior at zero cost. Under a 0.1 s deadline, FedRAS delivers 22% more timely updates and 23% fewer bytes than fixed rank 16 over ten seeds (p=0.002), while a full-model update misses every stress deadline and spends 58.7× the bytes. A pre-registered TOST equivalence test confirms that accuracy is preserved. The dynamic program improves on greedy scheduling by 28 to 55% when the budget binds, and by up to 64% over value-per-byte pricing, at 23–421 ms solver cost, and a deliberately miscalibrated delivery predictor costs at most

*Corresponding author
Email address: hutaondsc@163.com (Tao Hu)

1% of the assignment objective.

Keywords: Federated LoRA, Mobile edge computing, Rank selection, Deadline scheduling, Wireless federated learning

---

1. Introduction

Mobile edge networks increasingly host federated learning (FL) workloads in which heterogeneous clients such as smartphones, vehicles, UAV terminals, and satellite-connected devices collaborate on a shared model without exposing raw data [1, 2, 3, 4]. The dominant cost in such systems is the uplink. Full-parameter model updates saturate wireless capacity, drain client energy, and frequently miss round deadlines when links degrade during handover or outage, as reported for ground, aerial, and satellite segments [5, 6, 7, 8]. Low-rank adaptation (LoRA) has therefore become an attractive transmission format for FL, because only two small factor matrices per adapted layer are communicated instead of the full weight delta [9, 10, 11].

Existing Federated LoRA systems, however, treat the rank r as a model hyperparameter. The rank is selected from data and system constraints alone, possibly varying across clients [12, 13] or adapting across rounds [14, 15], but not from the state of a lossy, deadline-constrained link. This convention ignores a scheduling dimension that is directly visible to the network. The omitted coupling is mechanical. A larger rank transmits more bytes and therefore loses delivery probability exactly where it gains adaptation capacity, while a smaller rank travels light but forfeits capacity on the healthy links that could have carried it. Whether the extra capacity is worth the delivery risk depends on the link state, and no training-side rule observes that state.

This raises three questions. How should delivery probability depend on rank and observable link state? How should client–rank assignment be solved under budget and participation constraints? Does the resulting adaptation improve network behavior in a real trainer? These questions are answered in Sections 3, 4, and 5, respectively.

We answer these questions with a delivery model, a grid-based dynamic-programming solver, a rank-selection rule, and a two-tier evaluation. We call the method FedRAS. The contributions are as follows.

- **Deadline-aware client–rank model.** We formulate each round as a joint assignment in which admission and rank are selected together.

The model links rank to its real serialized payload, a delivery probability $P_{\text{on}}(r)$, and a saturating capacity gain $G(r)$. Section 3 develops the model and the scheduling formulation.

- **Dynamic-programming scheduler.** The estimated problem is a multiple-choice knapsack with an admission cap. We derive a pseudo-polynomial dynamic program that is exact for the discretized single-round estimated assignment problem, with complexity $O(|\mathcal{D}|KB_{\max}|\mathcal{R}|)$. The solver study measures greedy gaps and runtime.

- **Heterogeneous-rank aggregation.** We use rank-masked averaging with a shared frozen A factor, so only the trainable B factor is ever uploaded. Under this condition, the rule gives exact coordinate-wise averaging of active induced-update slices. Proposition 3 states the conditions and limits.

- **Two-tier evaluation at real payload sizes.** A real trainer covers non-IID CIFAR-10, CIFAR-100, and MNIST over ten seeds, and a 500-client scheduler simulation covers deadline, budget, mobility, and scale regimes. Under a 0.1 s deadline, FedRAS delivers 22% more timely updates and 23% fewer bytes than fixed rank 16 at equivalent accuracy, while full-parameter updates miss every stress deadline. The evaluation further covers external baselines, a candidate-pool decomposition of the scheduler's gains, a SAGIN link profile, and pre-registered equivalence tests.

We report accuracy, timely delivery ratio, and bytes per round as separate outcomes.

A preliminary poster version of this work studied joint client–protection scheduling for federated learning in space–air–ground mobile networks [16]. The present article substantially reformulates that line of work around a different schedulable resource. It replaces compression-and-protection policies with the LoRA rank as the per-client decision variable. It introduces the real-payload delivery hazard and the saturating capacity model of Section 3, replaces the heuristic admission-and-repair procedure with a dynamic program that is exact on the stated budget grid, and adds frozen-factor rank-masked aggregation for heterogeneous ranks. It also reports a new two-tier evaluation comprising a real federated LoRA trainer, a 500-client scheduling simulator, and a solver exactness study. Accordingly, all numerical results in

this article come from the revised formulation and implementation and are not comparable with the preliminary version.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 presents the system model and the scheduling formulation, Section 4 the solution method and heterogeneous-rank aggregation, and Section 5 the experimental evaluation. Section 6 discusses interpretation, deployment, and limitations, and Section 7 concludes.

2. Related work

2.1. Federated LoRA and heterogeneous-rank adaptation

LoRA freezes a pretrained backbone and learns low-rank increments $\Delta W = BA$ [9]. Federated LoRA trains these adapters across clients. Prior work has studied sparse updates and instruction tuning [10, 11], examined how standard parameter-efficient tuning behaves under the federated setting [17], and pursued personalization through model-heterogeneous or dual-adapter designs [18, 19]. A further line lets the rank vary across clients and aligns heterogeneous ranks for aggregation, by rank self-pruning with sparsity-weighted averaging [12], stacking [20], zero-padded rank alignment [13], or SVD-based dynamic rank allocation [14].

FFA-LoRA [21] froze one factor at its random initialization and showed that this improves stability in privacy-preserving federated fine-tuning. We adopt the same frozen-factor property for a stronger reason. When the frozen factor is identical across clients, averaging the trainable factor gives exact coordinate-wise averaging of the active induced-update slices, as Proposition 3 shows, and each client uploads only that factor. It thereby removes the factor-rotation ambiguity that makes joint factor averaging differ from update averaging. Section 4.4 develops this aggregation.

SA-FedLoRA schedules rank over rounds without conditioning on link state [15]. FedSA-LoRA studies asymmetric factor aggregation with client-local parameters [22]. Across these systems, the rank responds to training progress, parameter importance, or device capability; in none of them is it a decision variable conditioned on the delivery state of a deadline-constrained link. We evaluate an annealing baseline and joint A/B averaging while making rank a per-round decision conditioned on delivery state and deadline.

2.2. Communication-efficient federated learning

A broad literature reduces FL communication through sparsification, quantization, compressed sensing, and over-the-air aggregation [23, 24, 25, 26, 27, 28]. These methods compress a given update. Our scheduler selects its rank before transmission, so rank directly affects payload and deadline feasibility. The proposed capacity and delivery models can therefore be combined with other codecs.

2.3. Client selection and deadline-aware scheduling

Client selection improves FL efficiency under data, latency, and energy heterogeneity. Oort ranks candidates by a utility that couples data value with system efficiency [29]; FedCS selects clients under explicit resource constraints [30]; TiFL groups clients into latency tiers to bound stragglers [31]; further designs target cost and freshness [32, 33, 34]. A second line makes participation deadline- and reliability-aware in mobile and non-terrestrial networks [5, 6, 7, 8, 35]: LESSON admits wireless clients at differentiated frequencies under a latency budget [36], and FedSN coordinates federated training across heterogeneous LEO satellite networks [37]. These methods generally treat update size as fixed: participation is decided for a given payload, not jointly with it. We include an Oort-style selector as a selection baseline. FedRAS chooses participation and update rank together, yielding a multiple-choice knapsack.

2.4. Research gap and positioning

Across these lines, a common interface is missing. The rank-adaptation methods of Section 2.1 decide the rank from training state, parameter importance, or device capability; the selection and deadline-aware methods of Section 2.3 decide participation for a fixed update size. Neither treats the LoRA rank as a schedulable network resource whose payload sets the delivery probability on a lossy link, and none formulates the joint client–rank assignment under byte-budget and deadline constraints. Table 1 positions FedRAS against the closest systems along these dimensions.

We do not claim each ingredient individually as new. Our contribution is their combination: the LoRA rank treated as a deadline-constrained network resource with an explicit payload-conditioned delivery hazard, a client–rank assignment that is exact on the stated budget grid with quantified greedy gaps and solver cost, and validation with a real federated trainer in which adapters are placed on every block of the backbone and all byte accounting

Table 1: Positioning against the closest federated LoRA and client-selection systems. Cell entries reflect the cited papers' own descriptions; "link-aware" means that a scheduling or adaptation decision conditions on network delivery state (rate, deadline, delivery probability) rather than on training progress or device capability alone; for fixed-rank systems the aggregation column is not applicable (—); n/r means not reported.

System	Rank decision	Link-aware	Het.-rank aggregation	Joint adm.+rank	Payload accounting	Real training
FFA-LoRA [21]	fixed	—	—	—	parameter count	✓
SLoRA [10]	fixed	—	—	—	parameter count	✓
HetLoRA [12]	per client/round	—	zero-padding, sparsity-weighted	—	parameter count	✓
HeLoRA [13]	per client	—	zero-padding, KD variants	n/r	parameter count	✓
FedARA [14]	per layer/round	—	threshold mask arbitration	—	parameter count	✓
SA-FedLoRA [15]	per round	—	n/r	—	parameter count	n/r
FedSA-LoRA [22]	fixed	—	—	—	parameter count	✓
Oort [29]	none (rank fixed)	runtime profile	—	✓(admission)	runtime profile	✓
FedRAS (ours)	per client/round	✓	frozen-A rank masking	✓(joint MCKP)	real serialized sizes	✓

uses real serialized sizes. The theoretical contribution is mechanism correctness rather than a learning-performance guarantee: exactness of the assignment on the stated grid (Proposition 1) and exactness of frozen-factor aggregation (Proposition 3), each with its scope conditions stated where it applies. The claim is conditional on the surveyed literature of Section 2 and on the evaluated setting.

3. System model and scheduling formulation

3.1. Mobile-edge federated learning architecture and round protocol

We consider a mobile edge network in which an edge aggregator serves a client set $\mathcal{D} = \mathcal{D}_g \cup \mathcal{D}_a \cup \mathcal{D}_s$ of ground clients $\mathcal{D}_g$, aerial and UAV clients $\mathcal{D}_a$, and satellite-connected clients $\mathcal{D}_s$. Ground links are comparatively stable, aerial links face frequent handover, and satellite links suffer long propagation delay, intermittent visibility, and outages [4, 5]. The aggregator holds the global model, schedules every round, and aggregates the timely updates.

Let w^t denote the frozen backbone broadcast at round t. Client i trains LoRA factors $A_{i,\ell}$ and $B_{i,\ell}$ for the adapted layers $\ell \in \mathcal{L}$, with effective weight change $\Delta W_{i,\ell} = B_{i,\ell} A_{i,\ell}$. A rank r update uses the first r columns of $B_{i,\ell}$ and the matching rows of the shared A. Fig. 1 shows the resulting round pipeline. In every round, the aggregator estimates the link states, constructs and filters the client–rank actions, and solves the assignment with the grid-based dynamic program of Section 4; the admitted clients train locally with their factors sliced to the assigned rank, so only the first r coordinates receive gradient updates, and upload before the deadline τ; the aggregator then aggregates the timely updates with data-size weights and updates its link-state

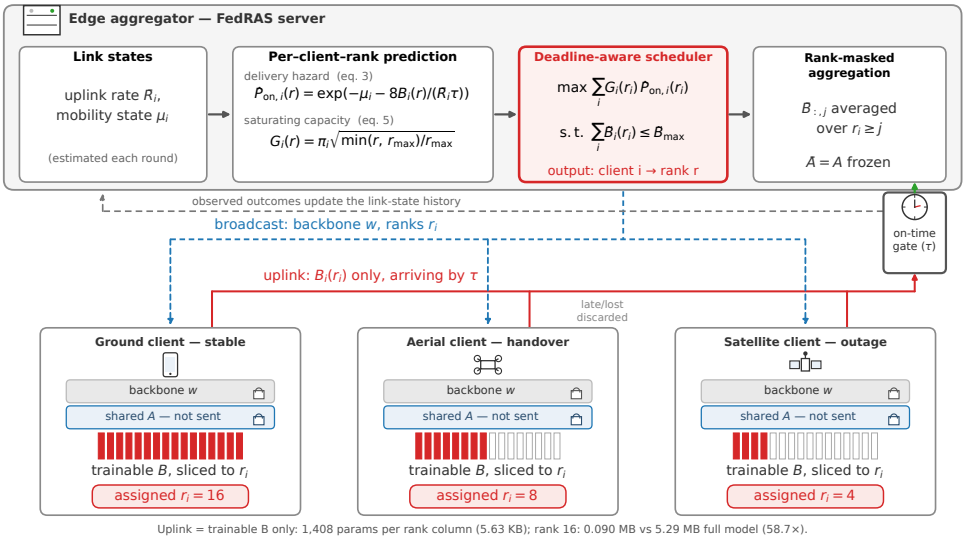


Figure 1: FedRAS system architecture and round pipeline. The edge aggregator scores every client–rank pair with the delivery hazard (3) and the saturating capacity (5), and computes the client–rank assignment with the grid-based dynamic program under the byte budget and the admission cap. Updates arriving within the deadline τ pass the on-time gate and are aggregated with rank-masked averaging while the shared factor A stays frozen; observed outcomes update the link-state history.

history from the observed outcomes. Late or lost updates are discarded, because stale updates harm convergence stability in deadline-critical federated learning. The experiments evaluate both a homogeneous link profile and a grouped ground–aerial–satellite profile with distinct rate and mobility laws (Section 5).

3.2. Link, mobility, and timely-delivery model

At decision time the scheduler receives an estimated uplink rate $\bar{R}_i^t$ in Mbps and a mobility state. The realized rate R_i^t, handover event $H_i^t \in \{0, 1\}$, and packet loss indicator J_i^t are not observed before transmission. For an update of $B_i^t(r)$ megabytes, the modeled delivery time is

$$T_i^t(r) = T_{\text{train,i}}(r) + T_{\text{proc,i}}(r) + \frac{8B_i^t(r)}{R_i^t} + d_i^t + H_i^t \Delta d_h. \tag{1}$$

Timely delivery is the event

$$P_{\text{on,i}}^t(r) = \Pr\left(T_i^t(r) \le \tau,\ J_i^t = 0 \mid x_i^t\right). \tag{2}$$

The implementation uses the following hazard surrogate rather than a derived distribution for all terms in (1).

$$\hat{P}^t_{\mathrm{on,i}}(r) = \exp\big(-\mu^t_i\big)\exp\Big(-\frac{8B^t_i(r)}{\bar{R}^t_i\tau}\Big). \tag{3}$$

The dimensionless intensity μ^t_i summarizes mobility-related failure risk, packet loss, handover effects, and unmodeled processing time; it is a fixed surrogate rather than an estimated distribution in the present implementation, and (3) is used as a plug-in predictor throughout. The construction of the estimated states and the sensitivity studies are specified in Section 5; how far the conclusions survive this idealization is delimited in Section 6.

3.3. Rank-dependent payload and adaptation capacity

With b bits per parameter and the frozen factor A shared and never transmitted, a rank-r client uploads only the trainable factor B, whose per-rank cost is the summed output dimension $D = \sum_{\ell\in\mathcal{L}} d_\ell$ of the adapted layers. The real serialized payload is

$$B^t_i(r) = \frac{b}{8}\, r\, D + B_{\mathrm{meta}}, \tag{4}$$

in bytes, with $D = 1{,}408$ parameters per rank column and $b = 32$ in the evaluated CNN, giving $B^t(16) = 0.090\,\mathrm{MB}$ when the header allowance B_{meta} is set to zero in the metering (Table 2). Rank is therefore a network-visible resource. It scales both the adaptation capacity and the transmission time. We model the useful contribution of a delivered rank-r update with the saturating capacity gain

$$G^t_i(r) = \pi^t_i\,\sqrt{\min(r, r_{\max})/r_{\max}}, \tag{5}$$

where π^t_i is a per-client contribution prior in the spirit of Oort-style utility [29]. The width $r_{\max}$ is the factor width provisioned at all clients. The square root encodes diminishing returns of rank; the trainer data support this shape, with the mean delivered update norm growing by about $1.9\times$ from rank 4 to rank 16 against the $4\times$ of a linear model. The saturation at $r_{\max}$ reflects that wider factors add payload but no expressible capacity once the provisioned width is reached. Full-parameter FedAvg updates are accounted at their true serialized size (Table 2).

3.4. Deadline-constrained client–rank scheduling formulation

Each round, the aggregator decides $a_i^t \in \{0\} \cup \mathcal{R}$, where $a_i^t = 0$ means client i does not participate and $a_i^t = r$ schedules it at rank r. The per-round problem is

$$\max_{\{a_i^t\}} \sum_{i \in \mathcal{D}} \mathbf{1}\{a_i^t > 0\}\, G_i^t(a_i^t)\, P_{\mathrm{on,i}}^t(a_i^t) \tag{6}$$

$$\text{s.t.} \quad \sum_i B_i^t(a_i^t) \le B_{\max}, \qquad \sum_i \mathbf{1}\{a_i^t > 0\} \le K. \tag{7}$$

The objective is a scheduling utility: expected useful delivered updates under the byte budget $B_{\max}$ and the participation cap K. It is a network-side objective, not a surrogate for final test accuracy; increasing r raises G sublinearly while raising payload and lowering the delivery surrogate. The experiments therefore report test accuracy, timely delivery ratio, bytes per round, and mean delivered rank as separate outcomes (Section 5).

4. Solution method and heterogeneous-rank aggregation

4.1. From formulation to discrete action space

Each round the aggregator selects, for every client, one action from the discrete set $a_i^t \in \{0\} \cup \mathcal{R}$ of Section 3.4. The per-rank payload follows (4), and the estimated utility of admitting client i at rank r is $u_i(r) = G_i^t(r)\hat{P}_{\mathrm{on,i}}^t(r)$. All budget arithmetic uses a conservative ceiling discretization onto a budget grid $q = 0.01\,\mathrm{MB}$, with integer cost $\widetilde{B}_i(r) = \lceil B_i^t(r)/q \rceil$, applied identically to the dynamic program, the greedy baselines, and the admission checks, so that every scheduling method optimizes over the identical feasible region. Under this discretization, the single-round estimated problem (6)–(7) is a multiple-choice knapsack with an admission constraint, for which an exact pseudo-polynomial dynamic program exists.

4.2. Grid-based dynamic-programming scheduler

Let $V_i(k, b)$ be the maximum estimated utility from the first i clients with k admissions and integer budget b. Initialize $V_0(0, 0) = 0$ and all other states to $-\infty$. The recurrence is

$$V_i(k,b) = \max\Big\{ V_{i-1}(k,b),\ \max_{r \in \mathcal{R}} \big[V_{i-1}(k-1, b - \widetilde{B}_i(r)) + u_i(r) \big] \Big\}, \tag{8}$$

where $u_i(r) = G_i^t(r)\hat{P}_{\mathrm{on,i}}^t(r)$. Backpointers recover the assignment. Algorithm 1 summarizes one scheduling round.

4.3. Correctness and complexity analysis

Proposition 1 (Per-round optimality)**.** *For every reachable state (k, b), recurrence* (8) *considers the skip action and every feasible rank for client i. Induction on i therefore proves exactness for the discretized single round estimated assignment problem. The result is not a claim of optimality for the continuous budget problem, the multi-round stochastic control problem, or final model accuracy.*

Proof. The admission indicator and the rank of each client are exclusive actions that affect the two resources additively. Admitting client i at rank r consumes one admission slot and $\widetilde{B}_i(r)$ budget units and adds $u_i(r)$ value, independent of the other clients. The state (k, b) therefore summarizes all interaction, and the standard interchange argument for the multiple-choice knapsack applies. Any optimal solution restricted to the first $i-1$ clients must itself be optimal for its state, which the recurrence maximizes. □

The complexity is $O(|\mathcal{D}|K\widetilde{B}_{\max}|\mathcal{R}|)$ time and $O(K\widetilde{B}_{\max})$ memory with rolling value arrays; the back-pointer table retained for reconstruction adds $O(|\mathcal{D}|K\widetilde{B}_{\max})$, where $\widetilde{B}_{\max} = \lceil B_{\max}/q \rceil$. Section 5.3.2 reports the resulting runtimes for 500 and 1000 clients. The exact formulation also yields a condition under which the cheap per-client rule of Section 3.4 is already optimal, which the trainer experiments of Section 5.2.5 rely on.

Proposition 2 (Cohort-local optimality of the per-client rule)**.** *Consider the single-round problem* (6)–(7) *restricted to a fixed candidate set, and suppose the all-$r_{\max}$ rank profile is budget-feasible, i.e., $\sum_i \widetilde{B}_i(r_{\max}) \le \widetilde{B}_{\max}$ over the candidates. Then the profile that assigns every admitted client its per-client maximizer $r_i^\star = \arg\max_{r\in\mathcal{R}} G_i(r)\hat{P}_{\mathrm{on},i}(r)$ is a feasible optimal solution of the restricted problem.*

Proof. Each candidate's cost is at most its $r_{\max}$ cost, so the per-client maximizer profile satisfies the budget whenever the all-$r_{\max}$ profile does, and it uses no more than the cap. Being feasible, it attains $\sum_i \max_r u_i(r)$, which upper-bounds the objective of every feasible profile; it is therefore optimal. By Proposition 1, the dynamic program returns an optimal profile and hence coincides with the per-client maximizer profile whenever the latter is unique. □

Algorithm 1 FedRAS scheduling round

Require: clients $\mathcal{D}$, ranks $\mathcal{R}$, cap K, budget $B_{\max}$, deadline τ, states $\{x_i^t\}$
Ensure: admitted set and assigned ranks

```
Estimate P̂^t_{on,i}(r) by (3) and G^t_i(r) by (5) for all i, r
Initialize V_0(0, 0) = 0; other DP states to −∞
for each client i ∈ D do
    for k = 0, ..., K and each budget state b do
        Apply skip and every feasible rank transition of (8)
        Store the maximizing action as a backpointer
    end for
end for
Backtrack from max_{k,b} V_{|D|}(k, b); return the client–rank assignment
Execute the round; aggregate timely rank-masked updates (Section 4.4);
update link-state history
```

4.4. Frozen-factor rank-masked aggregation

Clients scheduled at different ranks share an $r_{\max}$ coordinate system. For one adapted layer, client i has sample weight n_i and delivered rank r_i. The factor $A \in \mathbb{R}^{r_{\max} \times k}$ is frozen and shared, while $B_i \in \mathbb{R}^{d \times r_{\max}}$ is trainable. A rank r_i update activates columns 1 through r_i of B_i and the matching rows of A, giving $\Delta W_i = \sum_{j \le r_i} [B_i]_{:,j} [A]_{j,:}$.

Jointly trained factor pairs (A_i, B_i) are identifiable only up to invertible transforms, so separately averaged factors need not average the updates at all. The elevated-learning-rate ablation of Section 5 examines this aggregation rule empirically.

Rank-masked averaging. The server zero-pads inactive columns of each delivered B_i and averages each active coordinate over the clients that activate it,

$$\bar{B}_{:,j} = \frac{1}{W_j} \sum_{i:\, r_i \ge j} n_i \left[B_i\right]_{:,j}, \qquad W_j = \sum_{i:\, r_i \ge j} n_i, \tag{9}$$

while the frozen factor aggregates to itself, $\bar{A} = A$. The server then applies momentum per factor tensor. With $\Delta^t = \bar{B} - S^{t-1}$, the velocity and state update as $v^t = \beta\, v^{t-1} + (1-\beta)\, \Delta^t$ and $S^t = S^{t-1} + v^t$, with $\beta = 0.3$ damping client-sampling and link noise.

Proposition 3 (Exactness of frozen-factor aggregation)**.** *Under* (9)*, the j-th active rank slice of $\bar{B}A$ is the data-weighted average over clients with $r_i \ge j$.*

This is exact coordinate-wise averaging of active induced-update slices. It is not, in general, the unified data-weighted average of complete client updates. If a common denominator $W = \sum_i n_i$ is used instead of W_j, the unified average is recovered, but active high-rank slices are diluted by W_j/W. The two rules coincide for all possible updates when all delivered ranks are equal.

Proof. The first two parts follow from linearity. For frozen shared A, multiplying a weighted sum of B-coordinates by $\big[A\big]_{j,:}$ converts it into the same weighted sum of the matching update slices, and the two normalizers differ only in W_j versus W. The product $\bar{B}A = \sum_j \bar{B}_{:,j}\big[A\big]_{j,:}$ collects the slices. The third part holds because $W_j = W$ for every active j if and only if $r_i = \max_k r_k$ for all admitted i with $n_i > 0$. □

The proposition concerns active rank slices rather than whole-update equivalence. With shared frozen A, it gives coordinate-wise data-weighted averaging and avoids dilution caused by a common denominator. The elevated-learning-rate ablation tests the associated stability claim. It does not establish a general convergence guarantee.

5. Experimental evaluation

The evaluation tests accuracy and network behavior in a real trainer (Section 5.2), scheduling quality at 500 clients and solver cost (Section 5.3), and sensitivity to payload, predictor, capacity, and budget-grid assumptions (Section 5.4). All byte figures are real serialized sizes, and all deadline regimes are calibrated to those sizes.

5.1. Experimental setup

Real trainer. The trainer is implemented in PyTorch and runs on one A800 GPU. The backbone is a six-convolution BatchNorm CNN with channel progression 64/128/256, followed by two 256-wide linear layers. It is centrally pretrained with crop, flip, and erase augmentation, Nesterov momentum, weight decay, and cosine learning-rate decay, for 30 epochs on CIFAR-10, 45 on CIFAR-100, and 3 on MNIST, and then frozen. LoRA adapters with candidate rank set $\mathcal{R} = \{2, 4, 8, 16\}$ and provisioned width $r_{\max} = 16$ are placed on all six convolutions through 1×1 factors and on both head layers. A client assigned rank r trains with its factors sliced to the first r coordinates, so the rank genuinely bounds local adaptation capacity. The pretrained

backbone reaches a test accuracy of 0.914 on CIFAR-10, 0.693 on CIFAR-100, and 0.995 on MNIST.

Federated protocol and links. The real trainer uses 20 clients partitioned with Dirichlet $\alpha = 0.5$, 10 sampled participants per round, one local epoch, batch size 128, and 40 rounds. Local SGD uses learning rate 0.005, weight decay 10^{-4}, gradient clipping at 1, and cosine decay. The server uses rank-masked weighted averaging with momentum 0.3. Results use five seeds (0 through 4); the CIFAR-10 nominal and 0.1 s regimes, their external baselines, and the scheduling-arm comparison at the nominal deadline use ten seeds. Uplink rates are drawn from LogNormal$(2.5, 0.7)$ clipped to $[1, 40]$ Mbps and mobility from Beta$(2, 5)$, redrawn each round; a grouped ground–aerial–satellite profile with distinct rate and mobility laws per class is evaluated separately. These generated states are supplied as oracle estimates to the scheduler.

Deadline calibration. All deadline regimes are calibrated to the real payloads. The nominal round deadline is 2.0 s, at which transmission time consumes only 3% of a rank-16 update's delivery probability at the median 12 Mbps rate, while a full-model update already misses 83% of its deadlines. The stress deadlines are 0.1 s and 0.05 s, at which rank-16 timely delivery falls to 0.41 and 0.27 while rank-4 retains 0.62 and 0.57, so downshifting is genuinely load-bearing. A loose 10 s control restores full-model timely delivery to 0.52.

Model and payload audit. Table 2 audits the actual serialized sizes against the evaluated model. The frozen backbone has 1.28M parameters (5.12 MB in FP32). Each rank column carries 2,563 adapter parameters across the eight adapted layers, of which the frozen A contributes 1,155 and the transmitted B contributes 1,408, i.e., 5.63 KB per rank column. A rank-16 client therefore uploads 0.090 MB (FP32), and a full-model update transmits 5.29 MB, 58.7× more.

Simulator. The 500-client, 150-round scheduler simulation uses the same delivery and capacity models with uniform cohort sampling, per-client rank selection, and budget admission on the same ceiling grid. It covers nominal, deadline, budget, mobility, and 1000-client scalability regimes, with five seeds and common traces. Its accuracy column is a bounded scheduling proxy driven by delivered useful value, not a dataset accuracy; the full-update baseline receives the same capacity credit as rank 16 under the saturating model, a disclosed modeling assumption. A separate cell repeats the earlier stylized protocol ($s(r) = 0.02r + 0.5$ MB, rank-64 full-update stand-in, 0.25 s deadline,

Table 2: Payload audit of real serialized update sizes. Frozen-A clients upload only the trainable B factor (1,408 parameters per rank column); the A+B column is the joint-training equivalent. The stylized $s(r)$ column lists the protocol reused as a normalized-cost cell in Section 5.3.1.

Rank r	A+B FP32 (MB)	B-only FP32 (MB)	B-only FP16 (MB)	Stylized $s(r)$ (MB)
2	0.021	0.011	0.006	0.54
4	0.041	0.023	0.011	0.58
8	0.082	0.045	0.023	0.66
16	0.164	0.090	0.045	0.82
Full model	5.29 MB FP32 (58.7× the rank-16 B-only upload)			

12 MB budget) as a normalized-cost sensitivity check.

Scoring proxy. The simulator samples π_i uniformly from $[0.15, 0.45]$ and uses a bounded scheduling proxy rather than dataset accuracy. Starting at 0.1, it updates as $a^{t+1} = a^t + \eta\hat{g}^t(1-a^t) - \varepsilon a^t + \omega^t$, where $\hat{g}^t$ is useful delivered value normalized by K, $\eta = 0.08$, $\varepsilon = 0.0005$, and $\omega^t \sim \mathcal{N}(0, 5 \times 10^{-4})$ is a small tie-breaking noise.

Solver study. The solver study runs 30 instances per deadline and budget cell at 500 clients. It compares the grid-based DP with value-per-byte greedy, value greedy, a cohort-myopic rule, and fixed rank 16, and adds a client-count sweep that extends to 1000 clients. Budgets are 30 MB (slack), 4 MB, and 1.2 MB; the last admits only twelve rank-16 updates on the ceiling grid of the cap K=25 and is therefore binding.

Scheduling variants. Two deployment variants of the formulation are evaluated. In full grid scheduling, the aggregator solves (6) and (7) over all clients; the solver study evaluates this variant at 500-client scale, and a same-cohort variant restricts the dynamic program to a uniformly sampled cohort, isolating the value of candidate-pool widening from in-cohort rank allocation. In the real trainer, fair-cohort scheduling samples the fixed cohort of ten clients and applies the per-client rank rule $r_i^\star = \arg\max_r G_i^t(r) P_{\text{on,i}}^t(r)$. Section 5.2.5 compares the variants in the trainer.

All trainer methods use the same client partitions, local optimizer (the full-model baseline at a 0.4× learning-rate scale), rank set where applicable, generated rate and mobility states, deadline, and seed protocol. The FedAvg baseline is referred to as full-model updates throughout. Communication metrics count attempted uplink bytes, including late and failed uploads. No quantization, sparsification, energy, or fairness metric is claimed in this study.

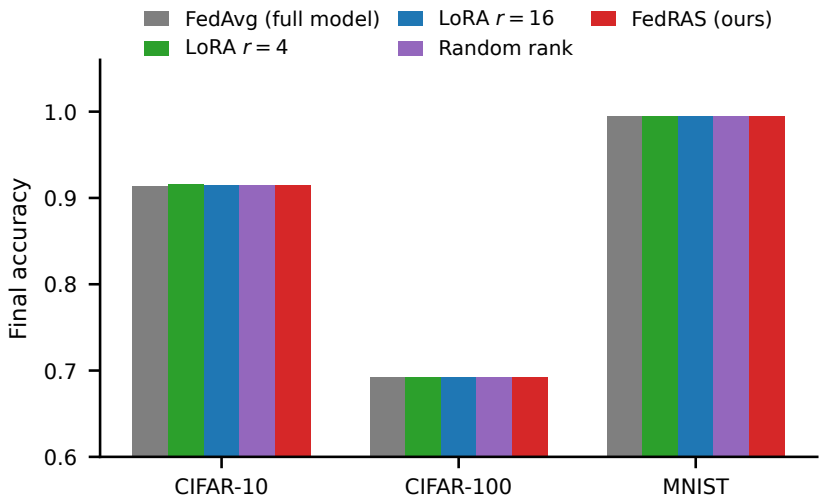


Figure 2: Final accuracy across datasets. CIFAR-10 uses ten seeds; CIFAR-100 and MNIST use five seeds. The parity and equivalence findings of Table 3 are not CIFAR-10-specific.

5.2. Evaluation with the federated LoRA trainer

5.2.1. Accuracy parity and rank invariance

Table 3 reports ten-seed means and standard deviations after 40 federated rounds on CIFAR-10. All LoRA methods hold the backbone accuracy within one standard deviation of 0.914, whether the rank is fixed at 4, fixed at 16, drawn at random, or selected by the rank rule. Full-parameter updates spend 58.7× the bytes of a rank-16 update, and they lose roughly three quarters of their timely deliveries, 0.17 against 0.73, because their 5.29 MB payloads rarely fit the round deadline. The cross-dataset results in Fig. 2 agree at 0.693 on CIFAR-100 and 0.995 on MNIST. At the nominal deadline the rank rule selects rank 16 for every client of every round, so FedRAS coincides with fixed rank 16 in bytes and timeliness, and in accuracy up to floating-point nondeterminism; this identity, rather than a tuned advantage, is what the nominal regime is meant to show. Fig. 3 shows the corresponding trajectories, stable over the 40 rounds for every rank policy.

5.2.2. Deadline stress

Table 4 tightens the deadline to 0.1 s and 0.05 s, with ten seeds at 0.1 s and five seeds at 0.05 s. Three effects emerge. First, FedRAS downshifts

Table 3: Real trainer on non-IID CIFAR-10 at the nominal 2.0 s deadline with ten seeds. Accuracy is measured at round 40. Timely delivery ratio is timely deliveries divided by selected clients. Bytes per round is the sum of attempted uplink payloads for selected clients over rounds 30 to 39, at real serialized sizes. Mean rank is computed over delivered updates.

Method	Accuracy	Timely delivery ratio	MB/round	Mean rank
FedAvg (full model)	0.9135 ± 0.0027	0.17	52.86	—
FedLoRA r=4	0.9154 ± 0.0016	0.75	0.23	4.0
FedLoRA r=16	0.9153 ± 0.0015	0.73	0.90	16.0
Random rank	0.9151 ± 0.0016	0.76	0.42	7.3
FedRAS (ours)	0.9153 ± 0.0016	0.73	0.90	16.0

Table 4: Real trainer under deadline stress. Accuracy is the round-40 mean and standard deviation. Timely delivery ratio, bytes per round, and mean delivered rank are averaged over rounds 30 to 39 at real serialized sizes. The 0.1 s columns use ten seeds; the 0.05 s columns use five seeds.

	deadline 0.1 s				deadline 0.05 s			
Method	Acc.	Timely	MB	Rank	Acc.	Timely	MB	Rank
FedAvg (full model)	$0.9153 \pm .0015$	0.00	52.86	—	$0.9147 \pm .0016$	0.00	52.86	—
FedLoRA r=4	$0.9154 \pm .0016$	0.62	0.23	4	$0.9144 \pm .0015$	0.57	0.23	4
FedLoRA r=16	$0.9149 \pm .0019$	0.41	0.90	16	$0.9143 \pm .0020$	0.27	0.90	16
Random rank	$0.9153 \pm .0017$	0.56	0.42	6.2	$0.9143 \pm .0015$	0.46	0.42	5.7
FedRAS (ours)	$0.9154 \pm .0018$	0.49	0.69	12.7	$0.9143 \pm .0014$	0.49	0.44	8.1

under pressure. Its delivered mean rank falls from 16.0 at the nominal deadline to 12.7 at 0.1 s and 8.1 at 0.05 s, where the delivery hazard (3) makes rank-16 payloads marginal. Second, downshifting pays. At 0.1 s, FedRAS delivers 0.49 timely updates against 0.41 for fixed rank 16 while transmitting 0.69 MB against 0.90 MB, i.e., 23% fewer bytes; at 0.05 s the gap widens to 0.49 against 0.27 timely deliveries at 51% fewer bytes. Third, full-parameter updates collapse completely. Their timely delivery ratio is 0.00 at both stress deadlines, so not a single full-size update arrives on time.

Accuracy spreads remain within about two standard deviations, which is consistent with the parity claim. The scheduling metrics are where the methods separate. Fig. 5 shows the ten-seed stress trajectories, and Fig. 4 visualizes the mechanism directly: the delivered rank of FedRAS holds 16 under the nominal deadline, tracks 13 under the 0.1 s deadline, and tracks 8 under the 0.05 s deadline, whereas the annealed baseline follows its fixed heat–cool path regardless of link state. A ten-seed replication confirms these differences at conventional significance levels, with the full statistics reported

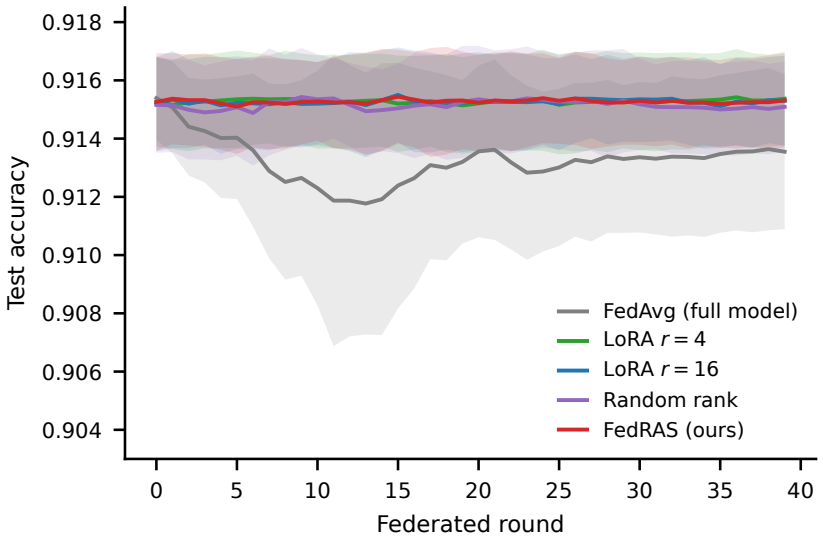


Figure 3: Real-trainer CIFAR-10 test accuracy over 40 federated rounds at the nominal deadline (mean over ten seeds). All rank policies hold the pretrained-backbone accuracy; differences appear in bytes and timeliness, not accuracy.

at the end of this subsection. Under the 0.1 s deadline, FedRAS transmits 23% fewer bytes than fixed r=16, with a seed-mean bootstrap 95% confidence interval from -26.6% to -20.6% and a two-sided Wilcoxon p of 0.002; it delivers a higher timely delivery ratio in ten of ten seed pairs, by 22% on average (p=0.002); and its accuracy is statistically equivalent within the pre-registered ± 0.5 percentage-point margin (TOST $p<10^{-6}$).

5.2.3. Extended stress matrix

Table 5 extends the study along further axes. A mobility-only scenario with Beta$(0.7, 5)$ mobility at the nominal deadline leaves all LoRA policies at parity, so mobility alone does not force downshifting at this deadline. Extreme label heterogeneity with α=0.1 reveals a second-order effect: rank-16 updates lose 0.32 percentage points relative to rank 4 (0.9107 versus 0.9139), consistent with capacity-amplified client drift, and full-parameter updates lose the most (0.9022). FedRAS follows the rank-16 trajectory here, so a heterogeneity-aware downshift is a natural extension. Loosening the deadline to 10 s restores full-model timely delivery to 0.52 without changing any LoRA accuracy. Repeating the 0.1 s deadline on CIFAR-100 and MNIST reproduces the CIFAR-10 stress picture. The scheduling conclusions are

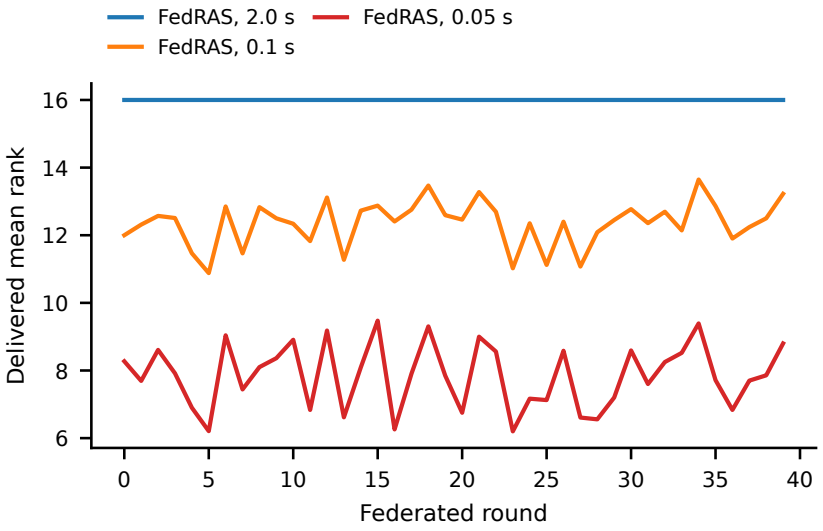


Figure 4: Delivered mean rank per round for FedRAS under three deadlines (ten seeds for 2.0 s and 0.1 s, five seeds for 0.05 s), against the fixed annealing schedule of the SA-FedLoRA-style baseline at 0.1 s. FedRAS is link-aware, holding high rank when possible and downshifting under pressure.

therefore not dataset-specific.

5.2.4. External baselines

Table 6 adds three external baselines to the trainer. The first is an SA-FedLoRA-style schedule that heats the rank from 2 to 16 and cools it down over the 40 rounds, oblivious to link state [15]. The second is an Oort-style selector that samples $2K$ candidates and admits the top-K by a data-size-and-recency utility at fixed rank [29]. The third is joint A/B training with plain averaging, the aggregation-level competitor of Section 4.4 [22].

The link-oblivious annealing schedule settles at low rank and therefore buys timely delivery with only 0.29 MB per round, but it cannot increase rank when links improve, and its capacity ceiling is rank 5. The Oort-style selector improves selection at fixed rank. Joint A/B training matches frozen-factor training at the adopted learning rate, and the ablations find no stability separation at an elevated learning rate either, so the case for the frozen factor rests on exactness and upload reduction rather than on an observed stability gap.

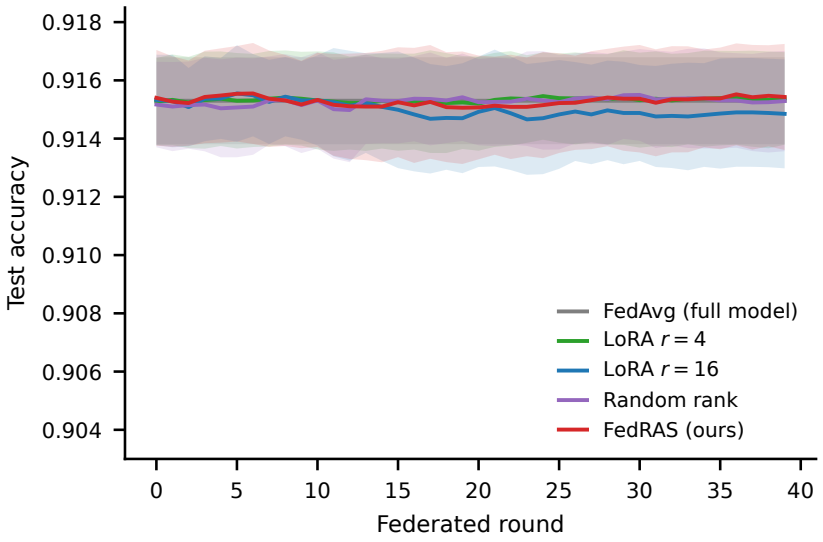


Figure 5: Real-trainer accuracy trajectories under the 0.1 s deadline (mean over ten seeds). All methods remain stable; full-model updates deliver nothing and stay at the backbone.

5.2.5. Scheduling variants

To attribute the value of exact scheduling, the three arms defined in the setup are compared in the real trainer, over ten seeds at the nominal deadline and five seeds under stress. The full grid arm solves the dynamic program of Section 4 over the entire 20-client population under the participation cap and a 1.0 MB budget, which equals ten rank-16 grid costs, so the budget binds only through downshifting or skipping. The same-cohort arm draws the identical cohort as fair cohort and restricts the same dynamic program to those candidates.

Table 7 yields a clean decomposition. At the nominal deadline, the same-cohort DP coincides exactly with the per-client argmax rule, as Proposition 2 predicts when the all-rank-16 profile fits the budget; the entire +9 percentage-point timely-delivery gain of the full DP (0.82 against 0.73) comes from admission over the wider candidate pool. Under the 0.1 s deadline the pattern persists: the in-cohort DP matches the argmax rule, while the full DP reaches 0.55 against 0.51 timely deliveries at 0.87 against 0.70 MB. Exactness therefore pays through admission control over a heterogeneous pool, not through rank allocation inside a cohort, where the cheap per-client rule is already optimal (Proposition 2). This delimits when full grid scheduling is worth its

Table 5: Extended stress matrix. Results use five seeds and report round-40 accuracy as mean over seeds. The mobility-only and α=1.0 scenarios probe extreme conditions; the last two columns repeat the 0.1 s deadline on CIFAR-100 and MNIST.

Scenario	FedAvg	r=4	r=16	Random	FedRAS
deadline 10 s	0.9134	0.9144	0.9145	0.9141	0.9143
mobility 0.7	0.9115	0.9143	0.9141	0.9145	0.9141
α=0.1	0.9022	0.9139	0.9107	0.9130	0.9109
α=1.0	0.9142	0.9143	0.9144	0.9141	0.9144
CIFAR-100, 2.0 s	0.6925	0.6928	0.6926	0.6926	0.6929
CIFAR-100, 0.1 s	0.6927	0.6927	0.6929	0.6927	0.6928
MNIST, 2.0 s	0.9947	0.9949	0.9951	0.9950	0.9951
MNIST, 0.1 s	0.9947	0.9949	0.9950	0.9949	0.9950

Table 6: External baselines in the real trainer on non-IID CIFAR-10 with ten seeds. Accuracy is measured at round 40. Timely delivery ratio, bytes, and delivered mean rank are averaged over rounds 30–39.

	nominal deadline				deadline 0.1 s			
Method	Acc.	Timely	MB	Rank	Acc.	Timely	MB	Rank
SA-FedLoRA-style annealing	$0.9154 \pm .0016$	0.75	0.29	5.1	$0.9153 \pm .0016$	0.62	0.29	5.0
Oort-style utility, fixed rank	$0.9150 \pm .0020$	0.76	0.45	8.0	$0.9149 \pm .0019$	0.54	0.45	8.0
Joint A/B, plain averaging	$0.9152 \pm .0016$	0.73	0.90	16	$0.9148 \pm .0019$	0.41	0.90	16
FedLoRA r=16	$0.9153 \pm .0015$	0.73	0.90	16	$0.9149 \pm .0019$	0.41	0.90	16
FedRAS (ours)	$0.9153 \pm .0016$	0.73	0.90	16.0	$0.9154 \pm .0018$	0.49	0.69	12.7

coordination cost.

5.2.6. Component ablations

Table 8 reports component ablations at the nominal deadline. In the parity regime, no removal measurably changes final accuracy, which is consistent with the claim that accuracy is carried by the backbone and not by the scheduler. The components instead shape scheduling behavior and aggregation correctness. Removing the capacity term collapses FedRAS to the minimum rank, the degenerate delivery-only policy that the capacity term exists to prevent; its capacity cost is invisible at nominal deadlines but material in the simulator, where reliability-only optimization loses 28 points of utility against rank 16 (Table 9). Removing server momentum is accuracy-neutral here. The rank mask itself is network-metric-neutral at fixed rank

Table 7: Scheduling arms in the real trainer. Top: nominal 2.0 s deadline over ten seeds. Bottom: 0.1 s deadline over five seeds. Both arms of each comparison share the candidate protocol described in the text.

Method	Accuracy	Timely	MB	Rank
nominal 2.0 s, ten seeds				
Full DP (grid assignment, 1.0 MB)	$0.9151 \pm .0016$	0.82	0.90	16.0
Same-cohort DP (1.0 MB)	$0.9152 \pm .0019$	0.73	0.90	16.0
Fair cohort (FedRAS rule)	$0.9153 \pm .0016$	0.73	0.90	16.0
FedLoRA r=16	$0.9153 \pm .0015$	0.73	0.90	16
stress 0.1 s, five seeds				
Full DP (grid assignment, 1.0 MB)	$0.9138 \pm .0011$	0.55	0.87	15.4
Same-cohort DP (1.0 MB)	$0.9139 \pm .0014$	0.51	0.70	12.7
Fair cohort (FedRAS rule)	$0.9144 \pm .0018$	0.51	0.70	13.0

policy: with the random-rank rule at the 0.1 s deadline, masked and unmasked aggregation give 0.9143 and 0.9145 final accuracy with identical byte and timeliness profiles. The mask is nonetheless what keeps heterogeneous-rank aggregation per-slice exact whenever ranks mix, as Proposition 3 states, and its absence dilutes high-rank coordinates under mixed-rank rounds.

Under the elevated learning rate of 0.02, the frozen-factor rule and joint A/B training are statistically indistinguishable, with final accuracies of 0.9130 and 0.9132 and transient minima of 0.913 and 0.912 across rounds and seeds. The stability separation between the two aggregation rules observed in our preliminary stylized-payload experiments does not survive the corrected implementation at this horizon, so the frozen-factor rule is justified by the exactness of Proposition 3 and by its single-factor uploads, not by an empirical stability margin at this scale.

5.3. Large-scale scheduling simulation

5.3.1. Scheduler behavior across operating regimes

Table 9 summarizes the simulator across regimes, with five seeds at the final round. The accuracy column is the bounded utility state of Section 5, not a dataset accuracy. In the nominal regime, FedRAS matches fixed rank 16 at 0.914 and dominates every low-rank policy. Under the 0.1 s deadline, FedRAS reaches 0.781 against 0.763 for fixed rank 16 at 21% fewer bytes, and full-model updates deliver nothing. Under the binding 1.2 MB budget,

Table 8: Component ablations, real trainer (non-IID CIFAR-10, five seeds, nominal deadline). The Min. column is the minimum over rounds of the per-round seed-mean accuracy in the elevated-learning-rate stability check at learning rate 0.02.

Configuration	Accuracy	Min.
Full system (ours)	$0.9144 \pm .0013$	—
w/o capacity term	$0.9145 \pm .0014$	—
w/o frozen A (joint A/B)	$0.9144 \pm .0015$	—
w/o rank mask (fixed random rank, 0.1 s)	$0.9145 \pm .0016$	—
w/o server momentum	$0.9143 \pm .0013$	—
frozen A, lr 0.02	$0.9130 \pm .0012$	0.913
joint A/B, lr 0.02	$0.9132 \pm .0012$	0.912

full-model updates cannot be admitted even once, while FedRAS and rank 16 tie at 0.797. The mobility-stress and 1000-client regimes show the same pattern. A final cell repeats the stylized protocol of the earlier literature: with the $s(r)$ payload model the ordering is preserved (FedRAS 0.413, rank 16 0.404, annealing 0.374), confirming that the conclusions are not an artifact of the payload accounting.

Fig. 6 shows the nominal and 0.1 s trajectories, and Fig. 7 the network-side view across regimes. Across the reported regimes, FedRAS is numerically best or tied for best; these simulator comparisons are descriptive unless an accompanying statistical test is reported.

5.3.2. Solver optimality and runtime

Table 10 reports the scheduling study with the identical ceiling discretization for the DP and every heuristic. When the byte budget is slack, value greedy is optimal and the fixed rank-16 rule is optimal whenever rank 16 is feasible for all cap clients, so the choice of scheduler matters little; the value-per-byte greedy, by contrast, collapses to a smallest-rank preference and loses up to 64% of the objective. The interesting regimes are the binding ones. When the budget binds at 1.2 MB, value-per-byte greedy loses 41 to 48%, value greedy 28 to 36%, and the cohort-myopic rule 48 to 55%. The DP reaches the optimum of the discretized single-round estimated problem in every cell. Its runtime is 23 to 421 ms for 500 clients depending on the budget-grid size, and it scales linearly to 1000 clients at 851 ms, as Fig. 8

Table 9: Scheduler simulation with 500 clients, 150 rounds, and five seeds. The accuracy column is a bounded scheduling proxy, not dataset accuracy. Timely delivery ratio is the mean delivery probability for admitted clients, and MB is the attempted uplink payload total for admitted clients in the final round. FedAvg refers to full-model updates at their real 5.29 MB size.

	Nominal 2.0 s			Deadline 0.1 s			Budget 1.2 MB		
Method	Acc.	Timely	MB	Acc.	Timely	MB	Acc.	Timely	MB
FedAvg (full)	.310	.34	26.5	.093	.00	26.5	.093	.00	0.0
r=4	.744	.75	0.8	.689	.64	0.8	.744	.75	0.8
r=16	.914	.73	2.5	.763	.42	2.5	.797	.76	1.2
Random	.816	.75	1.3	.709	.57	1.3	.803	.75	1.2
Reliability-only	.635	.76	0.5	.606	.70	0.5	.635	.76	0.5
SA-annealing	.842	.76	0.5	.730	.70	0.5	.796	.76	0.5
FedRAS (ours)	.914	.73	2.5	**.781**	.50	2.0	**.797**	.76	1.2

Table 10: Solver study with 500 clients, K=25, and 30 instances per cell, all methods on the identical ceiling grid. The table reports heuristic objective loss relative to the grid-based DP and DP runtime.

Deadline (s)	Budget (MB)	Greedy val/byte	Greedy value	Cohort myopic	Fixed r=16	DP time (ms)
2.0	30	63.6%	0.0%	40.9%	0.0%	421
2.0	4	63.6%	0.0%	40.9%	0.0%	51
2.0	1.2	47.8%	28.4%	48.9%	28.4%	23
1.0	30	62.7%	0.0%	40.0%	0.0%	415
1.0	4	62.7%	0.0%	43.0%	0.0%	51
1.0	1.2	47.3%	29.3%	48.3%	29.3%	23
0.1	30	52.8%	0.0%	51.9%	0.0%	415
0.1	4	52.6%	0.0%	52.0%	0.0%	51
0.1	1.2	41.0%	36.0%	54.8%	36.0%	23

shows. This is compatible with per-round scheduling at minute-scale FL rounds.

A dedicated grid sweep with steps from 0.02 to 0.005 MB compares each step against a 0.0025 MB reference. In the binding 1.2 MB regime the default 0.01 MB step deviates by at most 3.1%, and a 0.005 MB step bounds the deviation to 2.4%; slack regimes stay below 1%. The grid exactness claim of Section 4 therefore applies to a grid whose quantization error is bounded, measured, and adjustable at a known runtime cost.

5.3.3. Delivery-predictor sensitivity

The calibration study of Fig. 9 measures both the fidelity of the deployed predictor and the scheduling cost of its errors (500 clients, 150 round-

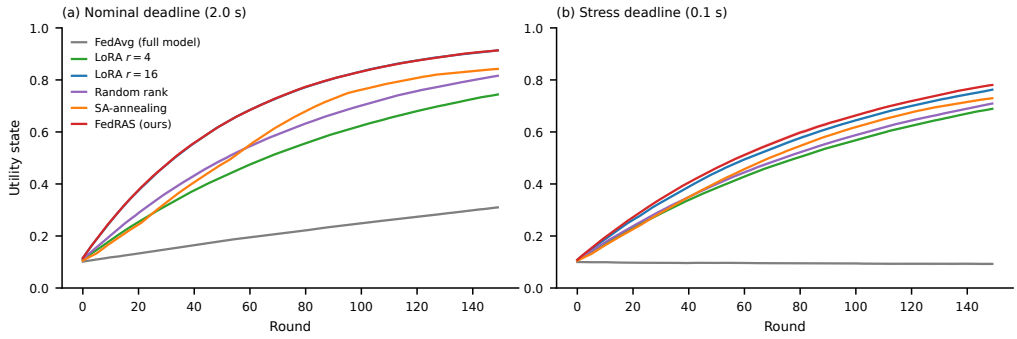


Figure 6: Scheduler simulation utility-state trajectories (mean over five seeds). (a) Nominal 2.0 s deadline, where FedRAS matches fixed rank 16. (b) 0.1 s deadline, where FedRAS downshifting separates from fixed rank 16.

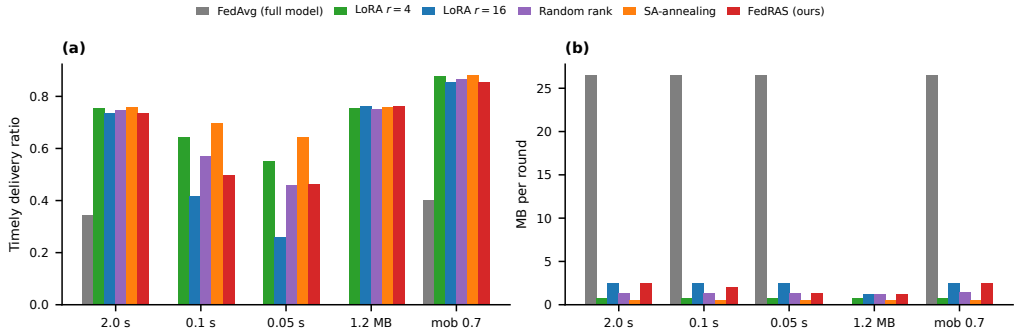


Figure 7: Timely delivery ratio and attempted bytes per method across simulator regimes (five-seed means, final round). Full-model updates deliver nothing under stress and cannot be admitted at all under the binding budget.

instances per cell). On the link model of Section 3, the deployed predictor with oracle link state is calibrated by construction, with an expected calibration error of 0.003 to 0.004. Deliberately mis-calibrated variants probe the consequences: power distortions $\hat{P}^{\gamma}$ with γ of 0.5 and 2 reach an expected calibration error up to 0.22, and multiplicative lognormal noise with $\sigma = 0.2$ injected into the predicted $\hat{P}_{\text{on}}$ up to 0.078. In every cell the resulting assignment-objective loss, evaluated under the true probabilities, stays below 1%, and its placement follows the structure of the error: the γ-distortions cost more under the stress deadline (up to 0.45%), where rank ordering matters most, while predictor noise costs the most in the nominal slack cell (0.95%), where it distorts an otherwise near-uniform rank preference. Mis-prediction

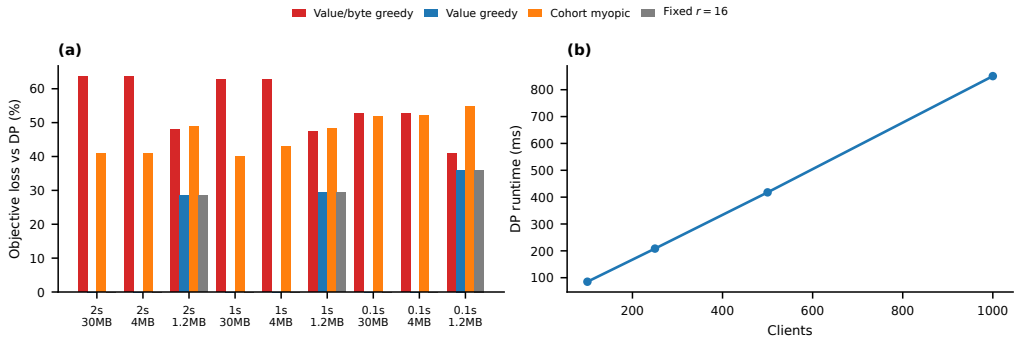


Figure 8: Solver study. (a) Objective loss of the heuristics relative to the DP across deadline and budget cells. (b) DP runtime scaling with the client population.

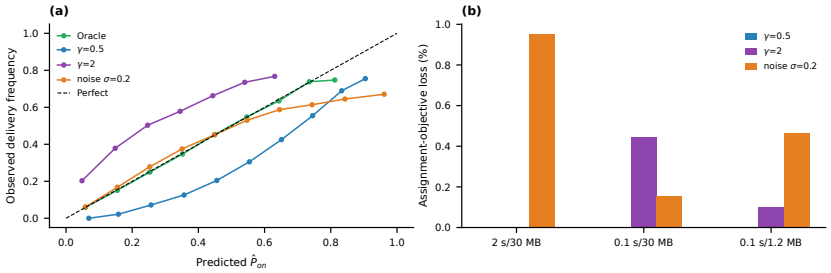


Figure 9: Delivery-predictor calibration study (500 clients, 150 round-instances per cell). The left panel shows reliability at the 0.1 s deadline. With oracle link state the hazard-form predictor is calibrated by construction, while the power-distorted (γ) and noisy variants are measurably miscalibrated. The right panel shows the assignment-objective loss of optimizing against the distorted predictor, evaluated under true probabilities, at most 1% in every cell.

therefore degrades the objective smoothly rather than catastrophically, and the observed delivery outcomes of Algorithm 1 are exactly the signal from which a per-client predictor would be recalibrated. Calibration on measured traces remains open and is taken up in Section 6.

5.4. Sensitivity and robustness analysis

Fig. 10 varies the provisioned adapter width, the cohort size, and the label heterogeneity of the trainer on CIFAR-10. Each variant uses five seeds; the nominal reference column uses ten. Widening the adapter from $r_{\max} = 8$ to 32 leaves accuracy flat while bytes rise from 0.45 to 1.80 MB and timely delivery falls from 0.75 to 0.70; the provisioned width is a network-facing

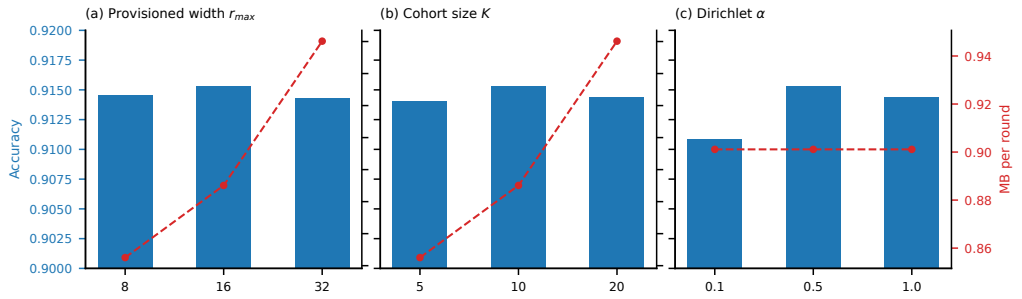


Figure 10: Sensitivity of the trainer to provisioned adapter width, cohort size, and label heterogeneity on CIFAR-10 with FedRAS (five seeds per variant; the nominal reference column uses ten seeds). Accuracy is flat in $r_{\max}$, cohort size, and α across the tested ranges; their effect is on bytes and delivered updates (red, right axis).

decision rather than an accuracy control. Cohort size from 5 to 20 leaves accuracy flat at linearly scaled bytes. Data heterogeneity from $\alpha = 0.1$ to 1.0 moves accuracy as Table 5 shows at constant network cost. Per-decision scheduling cost is negligible in all settings: the per-client rank rule costs about 0.1 ms per decision, roughly 1 ms per round, against 23 to 421 ms for the full grid-scheduling DP of Section 5.3.2.

SAGIN link profile. Grouping the 20 clients into ground, aerial, and satellite classes with distinct rate and mobility laws reproduces the aggregate parity at the nominal deadline (accuracy 0.9141) while exposing the expected per-class structure: timely delivery is 0.71 for ground, 0.61 for aerial, and 0.49 for satellite clients. At this deadline the scheduler holds rank 16 across all classes, so the class gap reflects the link laws alone rather than rank allocation. This profile is a parameterized link model, not a measured trace; its role is to show that the scheduler responds sensibly to client-class heterogeneity.

Taken together, the two evaluation tiers give a consistent picture. Rank choice does not measurably change accuracy for the frozen backbone at the 40-round horizon, and every accuracy comparison against fixed rank 16 passes the pre-registered TOST equivalence test at the ± 0.5 percentage-point margin; an offline control without delivery modeling trains close to the backbone level, separating learning capacity from delivery feasibility. The methods instead separate on the network side: FedRAS holds high ranks on healthy links and downshifts under pressure, full-parameter updates collapse once deadlines bind at real payload sizes, and the exact scheduler pays through admission over the candidate pool rather than in-cohort rank allocation (Propo-

sition 2). These tests rest on ten-seed replications of the headline trainer regimes with common random numbers within each seed; they support the stated network-side comparisons and the accuracy equivalence, and they do not establish accuracy dominance. Section 6 interprets these findings, states their scope conditions, and translates them into deployment guidance.

6. Discussion and limitations

6.1. Interpretation and deployment considerations

Why the gains are network-side. The trainer demonstrates parity and equivalence, not rank-dependent accuracy gains. With a strong frozen backbone and 40 adaptation rounds, transmitted rank does not measurably change test accuracy, because the accuracy is carried by the pretrained features while the scheduler shapes which updates arrive on time and at what byte cost. Claims that low rank limits adaptation quality would require longer horizons, weaker backbones, or distribution-shifted clients.

When FedRAS does not help. Three settings bound the method's value. First, when links are healthy and the budget accommodates every candidate at the provisioned width, FedRAS reduces exactly to fixed rank 16 (Proposition 2): the scheduling layer adds coordination cost and nothing else. FedRAS is therefore worth its coordination cost when deadlines are tight relative to real payloads or when the byte budget binds; outside those regimes, fixed rank 16 is the identical and simpler choice. Second, under extreme label heterogeneity (α=0.1), higher ranks lose 0.32 percentage points of accuracy relative to rank 4 through capacity-amplified client drift, and FedRAS follows the rank-16 trajectory and inherits that loss; conditioning the capacity term $G(r)$ on measured heterogeneity is left as future work. Third, under tight deadlines the weakest client class still delivers least (satellite 0.49 in the grouped profile), because downshifting protects delivery but cannot create link capacity.

Deployment recipe. The results translate into three planning rules for a practitioner. Calibrate the deadline ladder from the real payload of the widest provisioned rank, so that the nominal deadline leaves rank $r_{\max}$ near-constraint-free while at least one stress deadline drives its timely delivery below one half. Size the byte budget at ten times the grid cost of rank $r_{\max}$, which forces the budget to bind only through downshifting or skipping. Finally, by Proposition 2, use the per-client argmax rule when the budget accommodates every candidate at rank $r_{\max}$ and invoke the full dynamic

program only when the pool is heterogeneous enough that admission control matters.

6.2. Scope and limitations

Scope of claims. The results describe network-side gains in the evaluated frozen-backbone setting; they must not be read as universal accuracy gains. The simulator capacity model is a scheduling abstraction, and its accuracy column is a bounded scheduling proxy rather than a measured dataset accuracy.

Backbone pretraining. The centralized pretraining stage is a documented premise inherited from parameter-efficient FL practice; the federated phase adapts but does not train the backbone. Where no server-side corpus exists, self-supervised or public-data pretraining would be needed, and the parity result should be re-verified.

Estimator assumptions. The hazard-form delivery model (3) assumes the closed-form exponential realization, and the experiments supply generated link states as oracle estimates; training and processing times are included in the conceptual delivery model (1) but are not separately measured. Production deployments should calibrate P_{on} per client from observed outcomes, for example with isotonic or Platt scaling monitored by a Brier score, and the scheduler treats the predictor as a plug-in. Calibration on measured wireless traces remains open.

Myopia. The DP is exact for the estimated per-round problem, not for the multi-round stochastic control problem induced by budget carry-over and mobility correlation; a horizon-aware policy could shift admissions across rounds and is a natural extension.

Security. This work schedules communication and does not propose protection, so we make no privacy claim. Frozen-factor training incidentally reduces the dimensionality of transmitted gradients, which shrinks the gradient-inversion surface. Formal statements require differential-privacy accounting or secure aggregation, which compose orthogonally with rank scheduling [38, 39].

Generality. The trainer validates adapters on convolutional and linear layers of a CNN at $r_{\max}$=16 with 20 clients, and the simulator extends the population to 1000, both under synthetic link distributions. Extension to transformer backbones, measured rather than synthetic links, and larger rank sets may change the payload slope, capacity curve, and empirical conclusions. The formulation can be tested in those settings. The study does not measure

energy, system overhead beyond scheduler runtime, or client fairness and therefore makes no claim about those properties.

7. Conclusion

This paper reframed Federated LoRA over mobile edge networks as a deadline-aware client–rank scheduling problem in which the LoRA rank is a network-visible resource accounted at its real serialized size. We derived a rank-conditioned delivery model, formulated the per-round assignment as a multiple-choice knapsack under budget and participation caps, solved the grid-discretized assignment with a pseudo-polynomial dynamic program, and stabilized heterogeneous-rank aggregation with frozen-factor rank-masked averaging in which only the trainable factor is ever transmitted.

Experiments with a real federated LoRA trainer and a 500-client scheduler simulation, with deadlines calibrated to real payloads, show that rank selection preserves backbone accuracy up to a pre-registered equivalence margin while reshaping the network side of the exchange: under ten-seed deadline stress, FedRAS delivers 22% more timely updates and 23% fewer bytes than fixed rank 16, and the exact scheduler earns its cost through admission over heterogeneous candidate pools rather than through in-cohort rank allocation. The claims are conditional on the frozen-backbone setting and synthetic link models: rank scheduling adds value when deadlines bind relative to real payloads or when the byte budget binds, and reduces to fixed-rank operation otherwise. Future work includes horizon-aware stochastic scheduling, calibrated per-client delivery predictors on measured traces, protection mechanisms composed with rank scheduling, and validation on transformer backbones.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

The authors thank the members of the Information Engineering University research group for many useful discussions and for hands-on assistance with the experimental implementation.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

CRediT authorship contribution statement

Wenyu Ma: Methodology, Software, Validation, Investigation, Writing – original draft. **Tao Hu:** Conceptualization, Formal analysis, Supervision, Writing – review and editing. **Xinlei Liu:** Data curation, Investigation, Visualization. All authors approved the submitted version and accept responsibility for the work.

Data availability

The reproducibility package accompanying this submission contains the complete source code (trainer, simulator, solver, and analysis scripts), per-run configuration files, per-seed result tables, and the raw training logs supporting every table and figure in this study; a permanent repository DOI will be supplied at publication.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used AI-assisted tools for language editing and formatting support. The authors reviewed and edited all generated material and take full responsibility for the content of the published article.

References

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, B. A. y Arcas, Communication-efficient learning of deep networks from decentralized data, in: Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS), 2017, pp. 1273–1282.

[2] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings, R. G. L.

D'Oliveira, H. Eichner, S. E. Rouayheb, D. Evans, J. Gardner, Z. Garrett, A. Gascón, B. Ghazi, P. B. Gibbons, M. Gruteser, Z. Harchaoui, C. He, L. He, Z. Huo, B. Hutchinson, J. Hsu, M. Jaggi, T. Javidi, G. Joshi, M. Khodak, J. Konečný, A. Korolova, F. Koushanfar, S. Koyejo, T. Lepoint, Y. Liu, P. Mittal, M. Mohri, R. Nock, A. Özgür, R. Pagh, M. Raykova, H. Qi, D. Ramage, R. Raskar, D. Song, W. Song, S. U. Stich, Z. Sun, A. T. Suresh, F. Tramèr, P. Vepakomma, J. Wang, L. Xiong, Z. Xu, Q. Yang, F. X. Yu, H. Yu, S. Zhao, Advances and open problems in federated learning, Foundations and Trends in Machine Learning 14 (1–2) (2021) 1–210. doi:10.1561/2200000083.

[3] J. Liu, Y. Shi, Z. M. Fadlullah, N. Kato, Space-air-ground integrated network: A survey, IEEE Communications Surveys & Tutorials 20 (4) (2018) 2714–2741. doi:10.1109/COMST.2018.2841996.

[4] S. Mahboob, L. Liu, Revolutionizing future connectivity: A contemporary survey on ai-empowered satellite-based non-terrestrial networks in 6g, IEEE Communications Surveys & Tutorials 26 (2) (2024) 1279–1321. doi:10.1109/COMST.2023.3347145.

[5] N. Razmi, B. Matthiesen, A. Dekorsy, P. Popovski, Ground-assisted federated learning in leo satellite constellations, IEEE Wireless Communications Letters 11 (4) (2022) 717–721. doi:10.1109/LWC.2022.3141120.

[6] F. Tang, C. Wen, X. Chen, N. Kato, Federated learning for intelligent transmission with space-air-ground integrated network toward 6g, IEEE Network 37 (2) (2023) 198–204. doi:10.1109/MNET.104.2100615.

[7] Y. Qin, Y. Yang, F. Tang, X. Yao, M. Zhao, N. Kato, Differentiated federated reinforcement learning based traffic offloading on space-air-ground integrated networks, IEEE Transactions on Mobile Computing 23 (12) (2024) 11000–11013. doi:10.1109/TMC.2024.3389011.

[8] M. Ribero, H. Vikalo, G. de Veciana, Federated learning under intermittent client availability and time-varying communication constraints, IEEE Journal of Selected Topics in Signal Processing 17 (1) (2023) 98–111. doi:10.1109/JSTSP.2022.3224590.

[9] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen, LoRA: Low-rank adaptation of large language models, in: International Conference on Learning Representations (ICLR), 2022.

[10] S. Babakniya, A. R. Elkordy, Y. H. Ezzeldin, Q. Liu, K.-B. Song, M. El-Khamy, S. Avestimehr, SLoRA: Federated parameter efficient fine-tuning of language models, in: International Workshop on Federated Learning in the Age of Foundation Models (FedLLM), 2023.

[11] J. Zhang, S. Vahidian, M. Kuo, C. Li, R. Zhang, T. Yu, G. Wang, Y. Zhang, Y. Chen, H. Li, Towards building the federated GPT: Federated instruction tuning, in: IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 2024.

[12] Y. J. Cho, L. Liu, Z. Xu, A. Fahrezi, G. Joshi, Heterogeneous LoRA for federated fine-tuning of on-device foundation models, in: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2024, pp. 12903–12913. doi:10.18653/v1/2024.emnlp-main.717.

[13] B. Fan, X. Su, S. Tarkoma, P. Hui, HeLoRA: LoRA-heterogeneous federated fine-tuning for foundation models, ACM Transactions on Internet Technology 25 (2) (2025) 11:1–11:22. doi:10.1145/3723877.

[14] F. Wu, J. Hu, G. Min, S. Wang, Adaptive rank allocation for federated parameter-efficient fine-tuning of language models, IEEE Transactions on Computers 75 (4) (2026) 1650–1663. doi:10.1109/TC.2026.3655161.

[15] Y. Yang, X. Liu, T. Gao, X. Xu, G. Wang, SA-FedLoRA: Adaptive parameter allocation for efficient federated learning with LoRA tuning, arXiv preprint arXiv:2405.09394 (2024).

[16] W. Ma, T. Hu, X. Liu, Mobility-predictive risk-budgeted protection selection for federated learning in space–air–ground mobile networks, in: Proceedings of the ACM International Conference on Modeling, Analysis and Simulation of Wireless and Mobile Systems (MSWiM), 2026, accepted poster paper; publication details to be updated when available.

[17] Z. Zhang, Y. Yang, Y. Dai, Q. Wang, Y. Yu, L. Qu, Z. Xu, FedPETuning: When federated learning meets the parameter-efficient tuning methods of pre-trained language models, in: Findings of the Association for Computational Linguistics: ACL 2023, 2023, pp. 9963–9977. doi:10.18653/v1/2023.findings-acl.632.

[18] L. Yi, H. Zhang, H. Yu, G. Wang, X. Liu, Q. Hu, pFedLoRA: Model-heterogeneous personalized federated learning with homogeneous low-rank adapter sharing on mobile edge devices, IEEE Transactions on Mobile Computing 25 (9) (2026) 13898–13911. doi:10.1109/TMC.2026.3674996.

[19] Y. Yang, G. Long, T. Shen, J. Jiang, M. Blumenstein, Dual-personalizing adapter for federated foundation models, in: Advances in Neural Information Processing Systems 37 (NeurIPS), 2024.

[20] Z. Wang, Z. Gu, J. Weng, X. Shang, J. Li, J. Li, K. Wan, FLoRA: Federated fine-tuning large language models with heterogeneous low-rank adaptation, in: Advances in Neural Information Processing Systems (NeurIPS), 2024.

[21] Y. Sun, Z. Li, Y. Li, B. Ding, Improving LoRA in privacy-preserving federated learning, in: International Conference on Learning Representations (ICLR), 2024.

[22] P. Guo, S. Zeng, Y. Wang, Y. Fang, W. Huang, S. Liu, D. Guo, Selective aggregation for low-rank adaptation in federated learning, in: International Conference on Learning Representations (ICLR), 2025.

[23] Y. Oh, N. Lee, Y.-S. Jeon, H. V. Poor, Communication-efficient federated learning via quantized compressed sensing, IEEE Transactions on Wireless Communications 22 (2) (2023) 1087–1100. doi:10.1109/TWC.2022.3201207.

[24] Y. Oh, Y.-S. Jeon, M. Chen, W. Saad, Fedvqcs: Federated learning via vector quantized compressed sensing, IEEE Transactions on Wireless Communications 23 (3) (2024) 1755–1770. doi:10.1109/TWC.2023.3291877.

[25] H. Liu, F. He, G. Cao, Communication-efficient federated learning for heterogeneous edge devices based on adaptive gradient quantization, in: Proceedings of IEEE INFOCOM, 2023, pp. 1–10. doi:10.1109/INFOCOM53939.2023.10228970.

[26] M. Kim, A. L. Swindlehurst, D. Park, Beamforming vector design and device selection in over-the-air federated learning, IEEE

Transactions on Wireless Communications 22 (11) (2023) 7464–7477. doi:10.1109/TWC.2023.3251339.

[27] A. Reisizadeh, A. Mokhtari, H. Hassani, A. Jadbabaie, R. Pedarsani, Fedpaq: A communication-efficient federated learning method with periodic averaging and quantization, in: Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics, Vol. 108 of Proceedings of Machine Learning Research, PMLR, 2020, pp. 2021–2031.

[28] Y. Lin, S. Han, H. Mao, Y. Wang, W. J. Dally, Deep gradient compression: Reducing the communication bandwidth for distributed training, in: International Conference on Learning Representations, 2018.

[29] F. Lai, X. Zhu, H. V. Madhyastha, M. Chowdhury, Oort: Efficient federated learning via guided participant selection, in: Proceedings of the 15th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2021, pp. 19–35.

[30] T. Nishio, R. Yonetani, Client selection for federated learning with heterogeneous resources in mobile edge, in: IEEE International Conference on Communications (ICC), 2019. doi:10.1109/ICC.2019.8761315.

[31] Z. Chai, A. Ali, S. Zawad, S. Truex, A. Anwar, N. Baracaldo, Y. Zhou, H. Ludwig, F. Yan, Y. Cheng, TiFL: A tier-based federated learning system, in: Proceedings of the 29th International Symposium on High-Performance Parallel and Distributed Computing (HPDC), 2020. doi:10.1145/3369583.3392686.

[32] X. Chen, X. Zhou, H. Zhang, M. Sun, H. V. Poor, Client selection for wireless federated learning with data and latency heterogeneity, IEEE Internet of Things Journal 11 (19) (2024) 32183–32196. doi:10.1109/JIOT.2024.3425757.

[33] L. Dong, Y. Zhou, L. Liu, Y. Qi, Y. Zhang, Age of information based client selection for wireless federated learning with diversified learning capabilities, IEEE Transactions on Mobile Computing 23 (12) (2024) 14934–14945. doi:10.1109/TMC.2024.3450549.

[34] B. Luo, X. Li, S. Wang, J. Huang, L. Tassiulas, Cost-effective federated learning in mobile edge networks, IEEE Journal on

Selected Areas in Communications 39 (12) (2021) 3606–3621. doi:10.1109/JSAC.2021.3118436.

[35] D.-J. Han, W. Fang, S. Hosseinalipour, M. Chiang, C. G. Brinton, Orchestrating federated learning in space-air-ground integrated networks: Adaptive data offloading and seamless handover, IEEE Journal on Selected Areas in Communications 42 (12) (2024) 3505–3520. doi:10.1109/JSAC.2024.3459090.

[36] L. Yu, X. Sun, R. Albelaihi, C. Yi, Latency-aware semi-synchronous client selection and model aggregation for wireless federated learning, Future Internet 15 (11) (2023) 352. doi:10.3390/fi15110352.

[37] Z. Lin, Z. Chen, Z. Fang, X. Chen, X. Wang, Y. Gao, FedSN: A federated learning framework over heterogeneous LEO satellite networks, IEEE Transactions on Mobile Computing 24 (3) (2025) 1293–1307. doi:10.1109/TMC.2024.3481275.

[38] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, K. Seth, Practical secure aggregation for privacy-preserving machine learning, in: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS), 2017, pp. 1175–1191. doi:10.1145/3133956.3133982.

[39] K. Wei, J. Li, C. Ma, M. Ding, W. Chen, J. Wu, M. Tao, H. V. Poor, Personalized federated learning with differential privacy and convergence guarantee, IEEE Transactions on Information Forensics and Security 18 (2023) 4488–4503. doi:10.1109/TIFS.2023.3293417.